

Nama : Ikhlas Putra Pambagyo
Instansi : UPN Veteran Jawa Timur
NPM : 23081010024

TUGAS LARAVEL 6

Link Repository PDF

<https://github.com/ikhlaspp/SIB-NF-Academy-Batch-4-Full-Stack-Developer>

Link Repository Project

<https://github.com/ikhlaspp/booksales-api-laravel>

1. Membuat middleware bernama `Admin` menggunakan perintah artisan yang bertugas sebagai penengah (interceptor) untuk mengecek hak akses/role dari pengguna HTTP Request. Middleware ini memastikan bahwa pengguna yang sedang login (`\$request->user()`) memiliki role bertitel `admin`. Jika bukan, akses ditolak dan disajikan JSON berisi pesan "Unauthorized or Forbidden" (403)

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class Admin
{
    /**
     * Handle an incoming request.
     *
     * @param Closure(Request): (Response) $next
     */
    public function handle(Request $request, Closure $next): Response
    {
        if ($request->user() && $request->user()->role === 'admin') {
            return $next($request);
        }

        return response()->json(['message' => 'Unauthorized or Forbidden'], 403);
    }
}
```

2. Mengatur routing terkait Author, Genre, dan Book menjadi dua buah kelompok hak akses kontrol menggunakan `apiResource()`

```
Route::apiResource('genres', GenreController::class)->only(['index', 'show']);
Route::apiResource('authors', AuthorController::class)->only(['index', 'show']);
Route::apiResource('books', BookController::class)->only(['index', 'show']);

Route::middleware(['auth:sanctum', Admin::class])->group(function () {
    Route::apiResource('genres', GenreController::class)->except(['index', 'show']);
    Route::apiResource('authors', AuthorController::class)->except(['index', 'show']);
    Route::apiResource('books', BookController::class)->except(['index', 'show']);
});
```

3. Membuat `AuthController` untuk menangani endpoint login (`POST /api/login`) sehingga sistem dapat menerbitkan token JWT bagi user berdasarkan kredensial (email & password) yang dicocokkan dengan data pada tabel `users`. Model `User` juga sudah dilengkapi dan didukung trait `HasApiTokens` dari Laravel Sanctum sehingga token `Bearer` siap digunakan.

```
<?php

namespace App\Http\Controllers;
use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Models\User;
use Illuminate\Support\Facades\Hash;
class AuthController extends Controller
{
    public function login(Request $request)
    {
        $request->validate([
            'email' => 'required|email',
            'password' => 'required',
        ]);

        $user = User::where('email', $request->email)->first();

        if (! $user || ! Hash::check($request->password, $user->password))
        {
            return response()->json([
                'message' => 'Kredensial tidak valid'
            ], 401);
        }

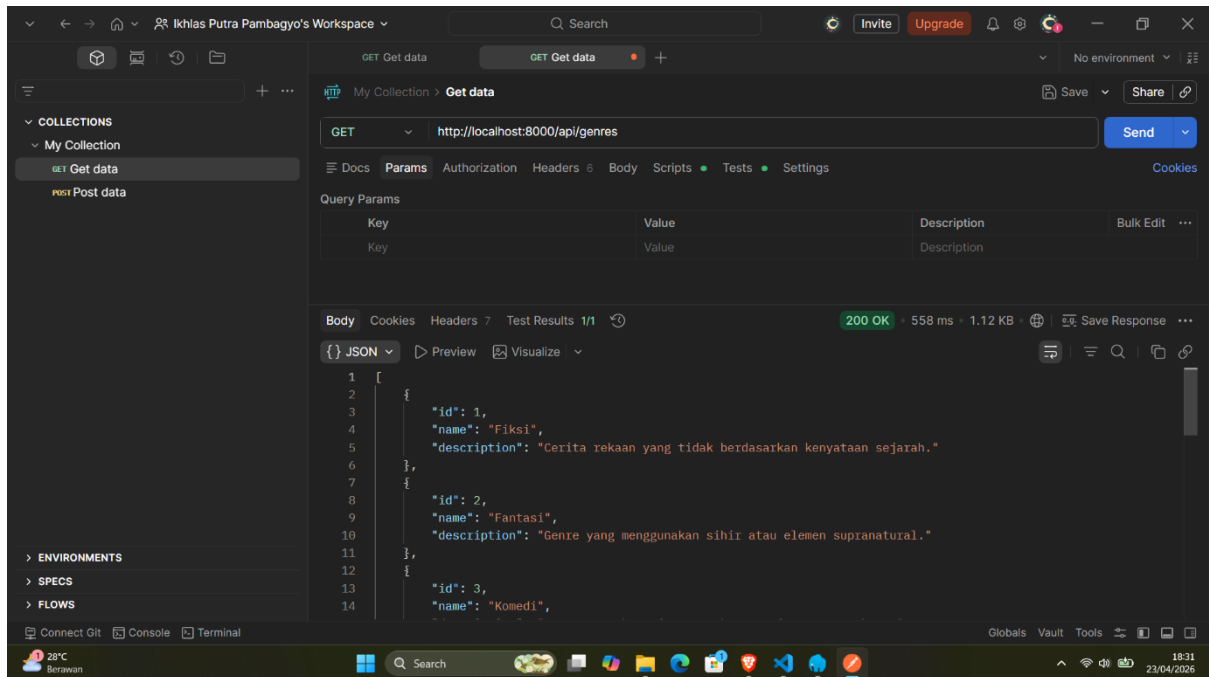
        $token = $user->createToken('auth_token')->plainTextToken;

        return response()->json([
            'message' => 'Login berhasil',
            'role' => $user->role,
            'access_token' => $token,
            'token_type' => 'Bearer',
        ]);
    }
}
```

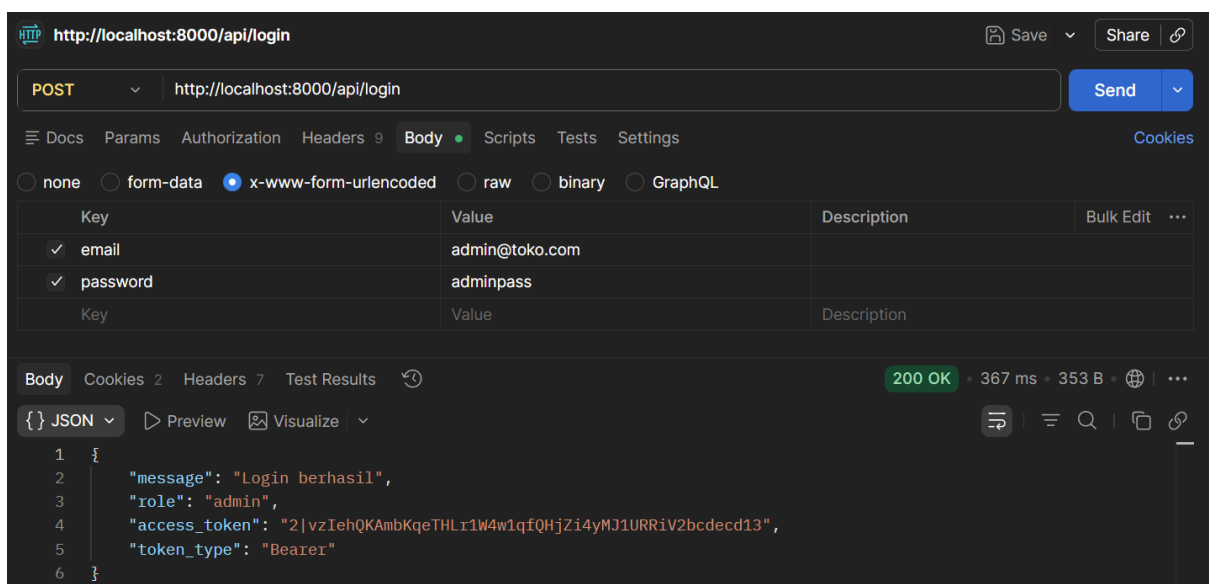
```
}  
}
```

Testing API menggunakan POSTMAN

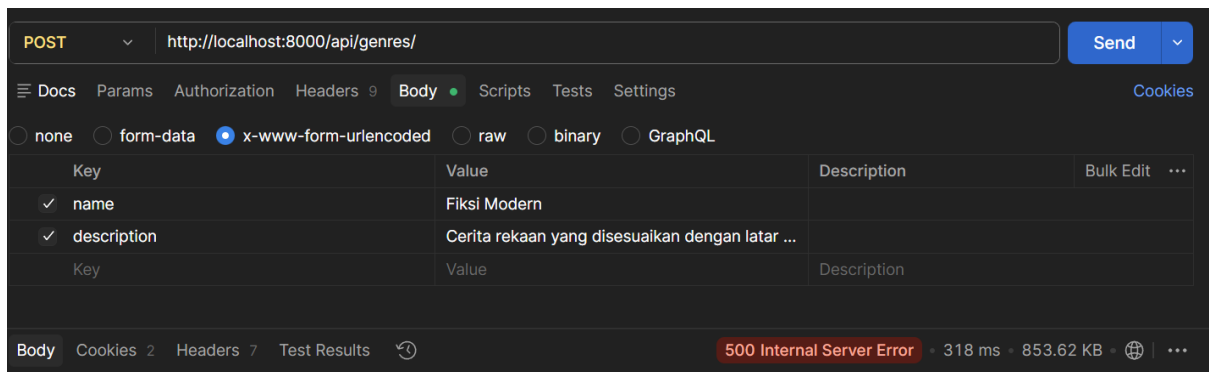
Menguji akses Read All & Show tanpa login



Mensimulasikan pengguna sebagai Admin via `'POST /api/login'`: Mendapatkan Token Akses masing-masing pihak.



Menguji API Create, Update, dan Destroy tanpa login



Menguji API Create, Update, dan Destroy dengan Token milik Admin

